

```
"""
```

```
verrouillage_1_5_v2_lyapunov_strict.py
```

```
=====
```

```
Contournement de la faille 3 du verrouillage 1.5 v1.
```

```
Augmentation de la statistique : 6 seeds au lieu de 3, production étendue 500N  
au lieu de 200N pour seeds originaux. Permet de :
```

- tester si seed 2 ($R^2=0.56$) était aberrant
- resserrer les barres d'erreur sur (λ , b)
- appliquer un critère strict $R^2 > 0.70$ pour rétention

```
Auto-audit intégré :
```

- rejet automatique seeds avec $R^2 < 0.70$
- rapport seeds rejetés vs gardés
- bootstrap sur la moyenne pour intervalle de confiance honnête

```
"""
```

```
import numpy as np
```

```
import math
```

```
import time
```

```
import json
```

```
from collections import deque
```

```
from scipy.spatial import KDTree
```

```
from scipy.stats import linregress
```

```
KAPPA_0 = 8
```

```
BETA = 0.30
```

```
TAU = 0.50
```

```
N_TEST = 200
```

```
BURN_FACTOR = 100
```

```
PROD_FACTOR = 500 # Étendu de 200 à 500
```

```
# Pipeline minimal (idem)
```

```
def _rips_radius(N, kappa=KAPPA_0): return (kappa / ((N-1)*(4/3)*math.pi))**(1/3)
```

```
def _E0(N, kappa=KAPPA_0): return int(N * kappa / 2)
```

```
def _lcc_size(adj, N):
```

```
    visited = [False]*N; best = 0
```

```
    for s in range(N):
```

```
        if not visited[s]:
```

```
            sz = 0; q = deque([s]); visited[s] = True
```

```
            while q:
```

```
                v = q.popleft(); sz += 1
```

```
                for w in adj[v]:
```

```

        if not visited[w]: visited[w] = True; q.append(w)
    best = max(best, sz)
return best

def _build_initial(N, seed):
    r = _rips_radius(N); r_p = 1.5 * r
    E0 = _E0(N)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K = sum(len(adj[i]) for i in range(N)) // 2
    att = 0; max_att = 10 * abs(E0 - K) + 5000
    while K != E0 and att < max_att:
        att += 1
        if K < E0:
            e = pool[rng2.integers(len(pool))]; u, v = e
            if v not in adj[u]: adj[u].add(v); adj[v].add(u); K += 1
        else:
            edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
            if not edges: break
            e = edges[rng2.integers(len(edges))]; u, v = e
            adj[u].discard(v); adj[v].discard(u)
            if _lcc_size(adj, N) == N: K -= 1
            else: adj[u].add(v); adj[v].add(u)
    return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

def compute_S(adj, n_tri, N):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    D = degs.std() / (degs.mean() + 1e-8)
    return math.log(1 + n_tri) - BETA * D

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:

```

```

    node = q.popleft()
    for w in adj[node]:
        if w in visited: continue
        if node == u and w == v: continue
        if node == v and w == u: continue
        visited.add(w); q.append(w)
        if w == v: return True
    return False

def run_mcmc_with_trace(pts, adj_init, pool, N, E0, seed, sample_every=None):
    if sample_every is None: sample_every = N
    tau_eff = TAU / N
    rng = np.random.default_rng(seed + 200_000)
    adj = [set(a) for a in adj_init]
    edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
    n_tri = _count_triangles(adj, N)
    S_curr = compute_S(adj, n_tri, N)
    n_steps = (BURN_FACTOR + PROD_FACTOR) * N
    pool_arr = np.array(pool, dtype=np.int32)
    n_pool = len(pool_arr)
    S_trace = []
    burn_steps = BURN_FACTOR * N
    for step in range(n_steps):
        if not edge_list: break
        idx_out = int(rng.integers(len(edge_list)))
        u, v = edge_list[idx_out]
        e_in = None
        for _ in range(30):
            idx_in = int(rng.integers(n_pool))
            a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
            if b not in adj[a]: e_in = (a, b); break
        if e_in is None: continue
        a, b = e_in
        if len(adj[u]) <= 2 or len(adj[v]) <= 2:
            if not _fast_still_connected(adj, u, v, N): continue
        tri_lost = len(adj[u] & adj[v])
        adj[u].discard(v); adj[v].discard(u)
        adj[a].add(b); adj[b].add(a)
        tri_gained = len(adj[a] & adj[b])
        n_tri_new = n_tri - tri_lost + tri_gained
        S_new = compute_S(adj, n_tri_new, N)
        dS = S_new - S_curr
        if dS >= 0 or rng.random() < math.exp(dS / tau_eff):
            n_tri = n_tri_new; S_curr = S_new
            edge_list[idx_out] = (a, b)
        else:

```

```

        adj[u].add(v); adj[v].add(u); adj[a].discard(b); adj[b].discard(a)
    if step >= burn_steps and step % sample_every == 0:
        S_trace.append(S_curr)
    return np.array(S_trace), S_curr

def estimate_lyapunov_with_audit(S_trace, n_burn_skip=50):
    """Estimation avec auto-audit strict."""
    trace = S_trace[n_burn_skip:]
    if len(trace) < 50:
        return {'status': 'insufficient_samples', 'n': len(trace)}
    S_max = trace.max()
    V_L = np.maximum(S_max - trace, 0)
    if V_L.std() < 1e-6:
        return {'status': 'V_L_too_concentrated'}
    V_t = V_L[:-1]; V_tp1 = V_L[1:]
    res = linregress(V_t, V_tp1)
    lambda_val = 1 - res.slope
    return {
        'status': 'OK',
        'lambda': float(lambda_val),
        'b': float(res.intercept),
        'a_slope': float(res.slope),
        'R2': float(res.rvalue**2),
        'n_samples': int(len(V_t)),
        'V_L_max': float(V_L.max()),
        'V_L_mean': float(V_L.mean()),
        'V_L_std': float(V_L.std()),
        'S_max_obs': float(S_max),
    }

if __name__ == "__main__":
    print("=" * 70)
    print("VERROUILLAGE 1.5 v2 - Foster-Lyapunov, statistique étendue")
    print("=" * 70)
    print(f"\nParamètres : N={N_TEST},  $\beta$ ={BETA},  $\tau$ ={TAU}")
    print(f"Burn-in {BURN_FACTOR}N, prod {PROD_FACTOR}N (étendu)")
    print(f"6 seeds, critère strict  $R^2 > 0.70$  pour rétention\n")

    seeds_to_test = [0, 1, 2, 3, 4, 5]
    all_results = []

    for seed in seeds_to_test:
        t0 = time.time()
        pts, adj_init, pool, E0 = _build_initial(N_TEST, seed)
        if _lcc_size(adj_init, N_TEST) / N_TEST < 0.9:

```

```

        print(f"Seed {seed}: init rejected (LCC < 0.9)")
        continue
    S_trace, _ = run_mcmc_with_trace(pts, adj_init, pool, N_TEST, E0, seed)
    t_mcmc = time.time() - t0
    est = estimate_lyapunov_with_audit(S_trace)
    if est.get('status') == 'OK':
        est['seed'] = seed
        est['wall_time_s'] = t_mcmc
        all_results.append(est)
        print(f"Seed {seed}:  $\lambda$  = {est['lambda']:.4f}, b = {est['b']:.5f}, "
              f" $R^2$  = {est['R2']:.3f}, n_samples = {est['n_samples']} [{t_mcm
    else:
        print(f"Seed {seed}: {est.get('status')}")

print(f"\n{'='*70}")
print("AUTO-AUDIT – Filtrage par  $R^2$  strict")
print(f"\n{'='*70}")

# Rétention stricte  $R^2 > 0.70$ 
R2_THRESHOLD = 0.70
kept = [r for r in all_results if r['R2'] >= R2_THRESHOLD]
rejected = [r for r in all_results if r['R2'] < R2_THRESHOLD]

print(f"\nSeeds totaux : {len(all_results)}")
print(f"Rétenus ( $R^2 \geq$  {R2_THRESHOLD}) : {len(kept)}")
print(f"Rejetés ( $R^2 <$  {R2_THRESHOLD}) : {len(rejected)}")

if rejected:
    print(f"  Seeds rejetés et raisons :")
    for r in rejected:
        print(f"    Seed {r['seed']}:  $R^2$  = {r['R2']:.3f} (en dessous du seuil

if kept:
    lambdas = np.array([r['lambda'] for r in kept])
    bs = np.array([r['b'] for r in kept])
    R2s = np.array([r['R2'] for r in kept])

    # Bootstrap pour intervalle de confiance honnête
    n_boot = 1000
    rng_boot = np.random.default_rng(42)
    boot_means_lambda = []
    for _ in range(n_boot):
        sample = rng_boot.choice(lambdas, size=len(lambdas), replace=True)
        boot_means_lambda.append(sample.mean())
    ci_low, ci_high = np.percentile(boot_means_lambda, [2.5, 97.5])

    print(f"\nRésultats finaux (après rétention) :")

```

```

print(f"  λ      = {lambdas.mean():.4f} ± {lambdas.std():.4f}")
print(f"  λ      IC 95% (bootstrap) : [{ci_low:.4f}, {ci_high:.4f}]")
print(f"  b      = {bs.mean():.5f} ± {bs.std():.5f}")
print(f"  R²      = {R2s.mean():.3f} ± {R2s.std():.3f}")

print(f"\n{'='*70}")
print("VERDICT v2 (sans complaisance)")
print(f"{'='*70}")

# Critères stricts
c1 = (lambdas > 0).all()
c2 = ci_low > 0
c3 = len(kept) >= 4
c4 = R2s.min() > 0.70

print(f"  ✓ λ > 0 sur tous seeds retenus      : {c1}")
print(f"  ✓ IC 95% λ exclut 0                    : {c2}   (low={ci_low:.4f})")
print(f"  ✓ Au moins 4 seeds retenus             : {c3}   (n_kept={len(kept)})")
print(f"  ✓ R² minimum > 0.70                   : {c4}   (R²_min={R2s.min():.3f})")

all_pass = c1 and c2 and c3 and c4

if all_pass:
    print(f"\n  → 4/4 critères stricts respectés.")
    print(f"  → STATUT FINAL 1.5 : [T] partiel solide (à scaler 20 seeds
else:
    n_pass = sum([c1, c2, c3, c4])
    print(f"\n  → {n_pass}/4 critères respectés.")
    if n_pass >= 3:
        print(f"  → STATUT FINAL 1.5 : [I] robuste (manque {4-n_pass} cri
    else:
        print(f"  → STATUT FINAL 1.5 : [I] indicatif, audit V108 nécessai

# Sauvegarde
with open('/home/claude/verrouillage_1_5_v2_results.json', 'w') as f:
    json.dump({
        'N': N_TEST, 'beta': BETA, 'tau': TAU,
        'burn_factor': BURN_FACTOR, 'prod_factor': PROD_FACTOR,
        'all_results': all_results,
        'kept_count': len(kept) if kept else 0,
        'rejected_count': len(rejected) if rejected else 0,
        'R2_threshold': R2_THRESHOLD,
    }, f, indent=2)
print(f"\n  Sauvegarde : /home/claude/verrouillage_1_5_v2_results.json")

```